

Лабораторная работа №13: «Тестирование и отладка REST API»

с использованием `TestClient`

Цель работы

Научиться разрабатывать автоматические тесты для веб-приложений, находить и исправлять логические ошибки в коде, используя подход **Test-Driven Debugging** (тесты сначала падают, потом исправляем код). В работе используется встроенный тестовый клиент FastAPI, который не требует запуска сервера и не создаёт сетевого трафика – поэтому **firewall не помеха**.

Исходные данные (предоставляются студентам)

Стек: Python 3.9+, FastAPI, Uvicorn, pytest, httpx (для `TestClient`).

Структура проекта:

```
todo_api/
├── main.py          # точка входа, описание эндпоинтов
├── models.py        # Pydantic-модели для запросов/ответов
├── database.py       # имитация базы данных (список задач в памяти)
└── requirements.txt
```

Содержимое файлов (код с намеренными багами)

requirements.txt

```
fastapi==0.104.1
uvicorn==0.24.0
pytest==7.4.3
httpx==0.25.1
```

database.py

```
# Имитация базы данных - просто список словарей
tasks_db = []
task_id_counter = 1

def get_all_tasks():
    return tasks_db

def get_task_by_id(task_id: int):
    for task in tasks_db:
        if task["id"] == task_id:
            return task
    return None # баг: не возвращает ошибку, а просто None
```

```

def add_task(task_data: dict):
    global task_id_counter
    new_task = {
        "id": task_id_counter,
        "title": task_data["title"],
        "description": task_data.get("description", ""),
        "completed": False,
        "created_at": "2026-09-07" # баг: дата всегда сегодняшняя,
не из запроса
    }
    tasks_db.append(new_task)
    task_id_counter += 1
    return new_task

def update_task(task_id: int, updated_data: dict):
    task = get_task_by_id(task_id)
    if task is None:
        return None
    # баг: не обновляет поле completed, если оно передано
    if "title" in updated_data:
        task["title"] = updated_data["title"]
    if "description" in updated_data:
        task["description"] = updated_data["description"]
    # забыли обработать "completed"
    return task

def delete_task(task_id: int):
    global tasks_db
    task = get_task_by_id(task_id)
    if task is None:
        return False
    tasks_db = [t for t in tasks_db if t["id"] != task_id]
    return True

def reset_db():
    """Сбрасывает состояние базы данных (для тестов)"""
    global tasks_db, task_id_counter
    tasks_db = []
    task_id_counter = 1

```

models.py

```

from pydantic import BaseModel
from typing import Optional

class TaskCreate(BaseModel):
    title: str
    description: Optional[str] = None

class TaskUpdate(BaseModel):
    title: Optional[str] = None
    description: Optional[str] = None
    completed: Optional[bool] = None

```

main.py

```
from fastapi import FastAPI, HTTPException, status
from database import (
    get_all_tasks, get_task_by_id, add_task,
    update_task, delete_task
)
from models import TaskCreate, TaskUpdate

app = FastAPI(title="To-Do API")

@app.get("/tasks")
def list_tasks():
    return get_all_tasks()

@app.post("/tasks", status_code=status.HTTP_201_CREATED)
def create_task(task: TaskCreate):
    # бар: нет валидации длины заголовка (может быть пустая строка)
    new_task = add_task(task.dict())
    return new_task

@app.get("/tasks/{task_id}")
def get_task(task_id: int):
    task = get_task_by_id(task_id)
    if task is None:
        # бар: ошибка 404 не выбрасывается, возвращается None ->
        # FastAPI упадёт с 500
        raise HTTPException(status_code=404, detail="Task not
found")
    return task

@app.put("/tasks/{task_id}")
def update_task_endpoint(task_id: int, updated: TaskUpdate):
    task = update_task(task_id, updated.dict(exclude_unset=True))
    if task is None:
        raise HTTPException(status_code=404, detail="Task not
found")
    return task

@app.delete("/tasks/{task_id}")
def delete_task_endpoint(task_id: int):
    success = delete_task(task_id)
    if not success:
        # бар: при удалении несуществующей задачи возвращается 204
        # вместо 404
        # (так как delete_task возвращает False, но мы не
        # обрабатываем это)
        pass
    return {"status": "deleted"} # всегда возвращает 200, даже
если задача не найдена
```

Задание

Часть 1. Подготовка

- Запустите PyCharm+ и создайте виртуальное окружение.
- Установите зависимости: `pip install -r requirements.txt`.
- Убедитесь, что файлы `main.py`, `models.py`, `database.py` лежат в одной папке.

Важно: сервер **не нужно** запускать отдельно – тесты будут использовать `TestClient` из библиотеки `fastapi.testclient`, который эмулирует HTTP-запросы внутри Python-процесса.

Часть 2. Написание автотестов

Создайте файл `test_api.py` в той же папке. Напишите тесты с использованием `pytest` и `TestClient`. Ваши тесты должны:

- Проверять **позитивные сценарии** (создание, получение, обновление, удаление).
- Проверять **негативные сценарии** (попытка получить несуществующую задачу, удалить несуществующую, создать с пустым заголовком и т.п.).
- **Обнаруживать все перечисленные баги** – тесты должны падать на текущей версии кода.

Рекомендуемая структура тестов:

- Фикстура `client`, возвращающая экземпляр `TestClient(app)`.
- Фикстура с `autouse=True`, которая перед каждым тестом вызывает `database.reset_db()` для очистки состояния.
- Тесты для каждого эндпоинта с использованием параметризации, где это уместно.

Пример (даётся в качестве подсказки):

```
import pytest
from fastapi.testclient import TestClient
from main import app
import database

@pytest.fixture
def client():
    return TestClient(app)

@pytest.fixture(autouse=True)
def clean_db():
    database.reset_db()
    # после теста ничего не делаем, так как reset_db уже очистила

def test_create_task(client):
    payload = {"title": "Купить молоко"}
    response = client.post("/tasks", json=payload)
```

```
assert response.status_code == 201
data = response.json()
assert data["title"] == "Купить молоко"
assert data["id"] == 1
assert data["completed"] is False
# далее другие проверки...

# ... здесь вы напишете остальные тесты
```

Вы должны написать минимум **8 тестов**, покрывающих все эндпоинты и все выявленные баги. Важно, чтобы тесты **падали** на текущей версии кода.

Совет: не забывайте проверять статус-коды и тело ответа. Для негативных сценариев проверяйте, что возвращается корректный HTTP-статус (например, 404 для несуществующей задачи).

Часть 3. Исправление багов

Найдите в исходном коде следующие дефекты (они специально внесены):

1. **GET /tasks/{id}** – если задача не найдена, приложение падает с Internal Server Error, а должно возвращать 404.
2. **POST /tasks** – не проверяется, что заголовок не пустой (можно создать задачу с `title=""`).
3. **PUT /tasks/{id}** – не обновляет поле `completed`, даже если оно передано в запросе.
4. **DELETE /tasks/{id}** – всегда возвращает 200 OK и сообщение `{"status": "deleted"}`, даже если задача не существовала; корректно – 404.
5. **Дополнительный баг (по желанию)** – дата создания всегда фиксированная ("2026-09-07"), а должна быть текущая дата (можно использовать `datetime.now().date()`).

Исправьте код в файлах `database.py` и `main.py` так, чтобы:

- Все ваши тесты проходили успешно.
- Логика работы соответствовала ожидаемой (см. описание эндпоинтов в Swagger, но Swagger в данном случае не открывается – ориентируйтесь на логику REST).

Часть 4. Отчёт

Подготовьте отчёт, в который включите:

- Листинг написанных тестов (`test_api.py`).
- Описание каждого найденного бага (где, в чём проявляется, как исправили).
- Скриншот успешного прохождения всех тестов (команда `pytest -v`).
- (По желанию) Предложение по улучшению качества кода (валидация, обработка ошибок, типизация).

Дополнительные указания

- Для запуска тестов просто выполните в терминале: `pytest -v` (сервер запускать не нужно!).
- При использовании `TestClient` все запросы выполняются синхронно, без реального сетевого взаимодействия.
- Для очистки базы данных между тестами используйте фикстуру `autouse`, как показано в примере. Это гарантирует независимость тестов.
- Если вы хотите проверить, что эндпоинт возвращает именно 404, используйте `assert response.status_code == 404`.
- Не забудьте импортировать `database` и вызывать `database.reset_db()` – эта функция уже добавлена в исходный код.
- Если у вас остались вопросы по работе с `TestClient`, обратитесь к официальной документации FastAPI (раздел "Testing").

Ожидаемый результат

- Папка с проектом, содержащая исправленный код (`main.py`, `database.py`) и файл `test_api.py`.
- Отчёт в формате PDF или docx.
- Все тесты зелёные (успешные).